Pharmacy Ordering System (InfoPharma) Software

IPOS-PU (Online Platform)

Summary

This is a Requirements Specification and High-Level Design Document, a proposal to the representatives of Info Pharma Ltd: Mr. Lancaster (founder and CEO), Mr Walter (CTO) and Mr Khan (Head of IT), by MediFlow Solutions. Henceforth, the representatives will be referred to as the **Client**.

Authors

Furqan Khokhar, Bahja Suliman, Dawud Shad, James Cahill, Haris Yasin, Huzefa Farooq, Faisal Jamsheer

MEDIFLOW

Contents

1 Preface.....3
1.1 Purpose and scope of the document.....3
1.2 Intended audience and reading suggestions.....3
2 Introduction.....3
3 Glossary.....4
4 System Requirements.....4
4.1 Purpose and scope of the system.....4
4.2 System models.....4
4.2.1 UML use-case models.....4
4.2.1.1 Use case diagram.....4
4.2.1.2 Use case specifications (simplified).....5
4.2.1.3 Prioritised list of use cases.....5
4.2.1.4 Class diagram (simplified).....9
5 Anticipated evolution of system requirements.....10
5.1 Integration with other IPOS subsystems.....10
5.2 User interface and customer experience.....10
5.3 Promotional features and merchant interaction.....10
6 IPOS high-level design.....10
6.1 Subsystems.....10
6.1.1 IPOS-SA.....10
6.1.2 IPOS-CA.....11
6.1.3 IPOS-PU.....11
6.2 Software architecture.....11
6.3 Design diagram of subsystem IPOS-PU.....12
6.4 Source code (Java) for the interfaces of IPOS-PU.....13
6.5 Testing the communications between the subsystems.....13
7 Appendices.....14
7.1 Initial statement of requirements by the client.....14
7.2 Hardware (minimal and optimal configuration).....14
7.2.1 Minimal hardware configuration.....14
7.2.2 Optimal hardware configuration.....14
7.3 RESTful API Best Practises.....15

1 Preface

1.1 Purpose and scope of the document

This is a Requirements Specification for one of the subsystems of the InfoPharma Ordering System (IPOS), IPOS-PU, and the and High-Level Design document for the InfoPharma Software, which includes IPOS-PU and two other subsystems, IPOS-SA and IPOS-CA. The document covers the functional requirement of IPOS-PU and the high-level design of the entire InfoPharma system showing the InfoPharma architecture composed of three subsystems and the mechanisms of interaction between them. The document is provided in response to the initial statement of requirements by Info Pharma Ltd, the Client.

The document explains what the subsystem shall do, and how its subsystems might interact with one another, but does not cover details about how the functionality of the IPOS-PU subsystem will be implemented. Various UML models are used together with fragments of software code to convey in more detail to system developers the expected behaviour of the IPOS-PU subsystem and especially how this subsystem will interact with the other subsystems of IPOS.

1.2 Intended audience and reading suggestions

The founder and CEO of InfoPharma Ltd., Mr Lancaster, and system analysis and architects responsible for the development of the IPOS system.

Some of the sections are intended for exclusively technical team members, which is clearly stated in the document prior to the section.

2 Introduction

IPOS will allow the Info Pharma Ltd to automate their business processes in three interrelated areas: the managing of product catalogue and stock, merchant sales and ordering supplies, and customer sales.

The three areas of planned automation are expected to be covered by three subsystems:

i. **IPOS-SA** A server application to maintain an electronic catalogue of pharmaceutical goods and control stock availability.
ii. **IPOS-CA** A client application run on a merchant's desktop to allow merchants to purchase goods and manage non-commercial customer sales.
iii. **IPOS-PU** An online platform to allow non-commercial customers to purchase goods from merchants and run promotions.

IPOS-PU will also be responsible for interfacing with integrating third-party services – SMTP, and Payment Processing, and it will provide these services to IPOS-SA and IPOS-CA as well.

Furthermore, IPOS-PU will generate updates for users on the status of their orders, as well as ensure that IPOS-CA is updated on changes to stock.

As well as this, IPOS-PU will generate reports at the request of an admin – these reports can be on promotional data or sales data.

3 Glossary

Key Term	Definition
Actor	A human or an external system interacting with IPOS, e.g., Admin, Payment Processor, etc.
IPOS	InfoPharma Ordering System Software
UML	Unified Modelling Language
Use case	A basic interaction of a system with an actor
Interface	A list of functions provided to an external system
Visual Paradigm	A software used to sketch UML diagrams.

4 System Requirements

4.1 Purpose and scope of the system

This section provides a more detailed specification of required functionality (known as system requirements) for IPOS-PU.

The intended audience are the *system architects, designers and software developers* responsible for the implementation of IPOS. Since we use UML, which requires a degree of specific expert knowledge, we do not expect this section to be immediately useful to the Info Pharma Ltd staff but would like to encourage them to review and comment, if possible.

4.2 System models

4.2.1 UML use-case models

4.2.1.1 Use case diagram

The use case diagram for the IPOS-PU subsystem is shown below in ***Figure 1***. In this diagram, the other subsystems of IPOS are treated as “external systems”, with which IPOS-PU may interact with – particularly when the subsystems require emails to be sent, or payments to be processed.

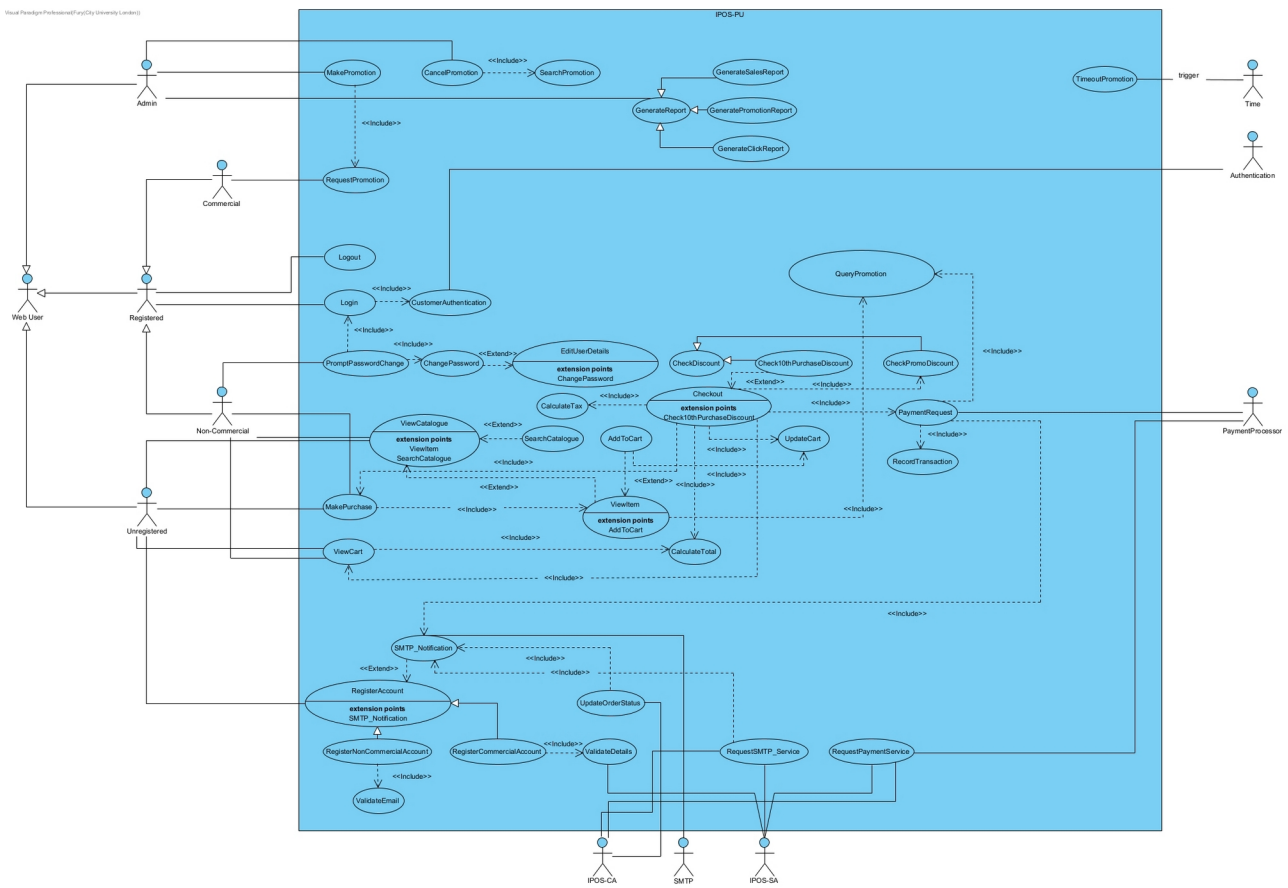


Figure 1*: Use case diagram of IPOS-PU. The use cases cover the required functionality (see the initial statement of requirements). The actors represent external entities (users or 3rd party systems) which interact with IPOS-PU. The other subsystems of IPOS are also modelled as actors.*

4.2.1.2 Use case specifications (simplified)

A complete set of ***simplified*** use case specifications are provided – detailed flows of interaction between actors and subsystems are not included.

Use case ID: UC01	**Use case:** MakePurchase
Brief Description: This use case facilitates the process where a customer selects items for purchase and moves toward finalising the transaction. It acts as a container for adding items to the basket and transitioning the customer into the formal checkout environment.	
Primary actors: Unregistered User, Non-commercial Customer	
Secondary actors: None	
Preconditions: • The customer has accessed the IPOS-PU catalogue. • The items intended for purchase are currently in stock within the IPOS-CA inventory.	
Main flow: Not required	
Extension use cases: None	
Inclusion use cases:	

1. ViewItem
Postconditions: 1) The customer is successfully transitioned to the Checkout interface. 2) The specific items and quantities are maintained in the active session for order confirmation.
Alternative flows: Empty basket Stock depletion

Use case ID: UC02	**Use case:** ViewItem
Brief Description: This use case allows a customer to view full details of a selected product from the catalogue, including name, description, price, availability, and any applicable promotions.	
Primary actors: Unregistered User, Non-commercial Customer	
Secondary actors: None	
Preconditions: 1) The IPOS-PU system is operational. 2) The product catalogue has been loaded and contains at least one item.	
Main flow: Not required	
Extension use cases: 1. AddToCart	
Inclusion use cases: 1. QueryPromotion	
Postconditions: 1) Product details are displayed to the customer. 2) If there is a promotion on that item, the customer can view it.	
Alternative flows: Item Not Found Item Removed/Obsolete	

Use case ID: UC03	**Use case:** ViewCatalogue
Brief Description: This use case allows any user to view the list of pharmaceutical goods offered by the merchant. The catalogue displays product names, descriptions, availability, and prices.	
Primary actors: Unregistered User, Non-commercial Customer	
Secondary actors: None	
Preconditions: 1) The IPOS-PU system is operational and has successfully loaded the product list from the local merchant database.	
Main flow: Not required	
Extension use cases: 1. ViewItem 2. SearchCatalogue	
Inclusion use cases: None	

Postconditions: 1) The user is presented with a list of products that are currently in stock or visible per merchant settings.
Alternative flows: No Products Found Out of Stock Filtering

Use case ID: UC04	**Use case:** SearchCatalogue
Brief Description: This use case allows the customer to search down the list of products by applying specific constraints, such as matching a keyword.	
Primary actors: Unregistered User, Non-commercial Customer	
Secondary actors: None	
Preconditions: 1) The IPOS-PU system is operational. 2) The user is currently viewing the product catalogue.	
Main flow: Not required	
Extension use cases: None	
Inclusion use cases: None	
Postconditions: 1) The catalogue view is updated to display only those products that match the customer's search criteria.	
Alternative flows: No Match Found	

Use case ID: UC05	**Use case:** Login
Brief Description: This use case explains how a registered user accesses member-only features by logging into the IPOS-PU system with credentials that was provided during registration.	
Primary actors: Registered Customer	
Secondary actors: None	
Preconditions: 1) The system is operational. 2) The user is registered and has valid login details.	
Extension use cases: None	
Inclusion use cases: 1. CustomerAuthentication	
Postconditions: 1) The user can access authorised features after successfully logging into IPOS-PU.	
Alternative flows: Invalid credentials	

Use case ID: UC06	**Use case:** Logout
Brief Description: This use case defines how a logged in user logs out of IPOS-PU.	
Primary actors: Registered Customer	

Secondary actors: None
Preconditions: • The system is operational. • The user is logged into IPOS-PU
Extension use cases: None
Inclusion use cases: None
Postconditions: 1) The user is successfully logged out of IPOS-PU.
Alternative flows: None

Use case ID: UC07	**Use case:** CustomerAuthentication
Brief Description: This use case verifies the identity of a registered user by checking their provided credentials (username/email and password) against the IPOS-PU database. It is a mandatory security step required before a user can access member features.	
Primary actors: Authentication Processor	
Secondary actors: Registered Customer	
Preconditions: 1) The user has navigated to the login screen. 2) The user has entered their unique username (email) and password.	
Main flow: Not required	
Extension use cases: None	
Inclusion use cases: None	
Postconditions: 1) The user's identity is verified, and the system proceeds to grant access to the portal (triggered by the Login use case).	
Alternative flows: Invalid Credentials Account Locked	

Use case ID: UC08	**Use case:** PromptPasswordChange
Brief Description: This use case triggers a mandatory password update interface immediately after a successful authentication. It is specifically designed for non-commercial members who have just logged in using their initial system-generated random password.	
Primary actors: Non-commercial Customer	
Secondary actors: None	
Preconditions: 1) The Login use case has been successfully completed. 2) The member's account is flagged for a mandatory password reset (e.g., first-time login).	
Main flow: Not required	

Extension use cases: None
Inclusion use cases: 1. ChangePassword 2. Login
Postconditions: 1) The user is redirected to the password change screen before any other portal functionality can be accessed. 2) The member is blocked from accessing the merchant catalogue or sales features until the password is updated.
Alternative flows: Authentication timeout User abort

Use case ID: UC09	**Use case:** ChangePassword
Brief Description: This use case allows a registered member to update their account password. For non-commercial members logging in for the first time, this process is mandatory and triggered automatically by the system.	
Primary actors: Registered User; Non-Commercial Customer (mandatory once)	
Secondary actors: None	
Preconditions: 1) The user has successfully authenticated through the Login and CustomerAuthentication use cases. 2) For first-time users, the system has identified the "force-change" status on the account.	
Main flow: Not required	
Extension use cases: None	
Inclusion use cases: None	
Postconditions: 1) The system-generated or old password is replaced with the new, user-defined password in the database. 2) The mandatory change flag is cleared, allowing full access to the portal.	
Alternative flows: Invalid Password Format Password Complexity Failure Abort Password Change	

Use case ID: UC10	**Use case:** EditUserDetails
Brief Description: This use case allows a registered member to modify their personal account information, such as their contact name or secondary contact details. It acts as a broader account management tool that can also trigger security updates.	
Primary actors: Registered User	
Secondary actors: None	
Preconditions: 1) The user is successfully logged into the IPOS-PU portal. 2) The user has navigated to the "Account Settings" or "Profile" section.	
Main flow: Not required	

Extension use cases: 1. ChangePassword
Inclusion use cases: None
Postconditions: 1) The updated user details are saved in the IPOS-PU member database. 2) A confirmation message is displayed to the user confirming the changes.
Alternative flows: Validation Error

Use case ID: UC11	**Use case:** ViewCart
Brief Description: This use case allows the user to view the current list of items they have selected for purchase. It provides a summary of product names, quantities, individual costs, and the cumulative total for the potential order.	
Primary actors: Unregistered User, Non-commercial Customer	
Secondary actors: None	
Preconditions: 1) The IPOS-PU portal is operational. 2) The user has added at least one item to their virtual shopping cart.	
Main flow: Not required	
Extension use cases: None	
Inclusion use cases: 1. CalculateTotal	
Postconditions: 1) The user is presented with a detailed summary of their cart contents. 2) The user has the option to proceed to the Checkout use case or return to ViewCatalogue.	
Alternative flows: Cart Empty Modify Quantity	

Use case ID: UC12	**Use case:** RegisterAccount
Brief Description: This use case allows an unregistered visitor to apply for membership on the online portal. The system captures the necessary details based on the selected account type and triggers an automated notification process to inform the applicant of the results.	
Primary actors: Unregistered User	
Secondary actors: None	
Preconditions: 1) The user is not currently logged into the IPOS-PU system. 2) The user has navigated to the registration section of the portal.	
Main flow: Not required	
Extension use cases: 1. SMTP_Notification	
Inclusion use cases: None	

Postconditions: 1) For non-commercial applicants, a new account is created and login details are sent via email. 2) For commercial applicants, the details are forwarded to IPOS-SA for manual review, and the user is notified that the application is pending.
Alternative flows: Invalid Details

Use case ID: UC13	**Use case:** RegisterCommercialAccount
Brief Description: This use case allows a business entity to apply for a commercial account.	
Primary actors: Unregistered User	
Secondary actors: IPOS-SA	
Preconditions: 1) The user has selected the "commercial" registration option. 2) The user has the necessary business details (Registration number, Director info, etc.)	
Main flow: Not required	
Extension use cases: None	
Inclusion use cases: 1. ValidateDetails	
Specialisation of: RegisterAccount	
Postconditions: 1) Application details are transmitted to the IPOS-SA subsystem. 2) The application status is set to "Pending" within IPOS-PU. 3) The user is notified via email that their application is under review.	
Alternative flows: Invalid Details	

Use case ID: UC14	**Use case:** RegisterNonCommercialAccount
Brief Description: This use case allows a member of the public to register for a personal account. The process requires no personal details other than a valid email address.	
Primary actors: Unregistered User	
Secondary actors: None	
Preconditions: 1) The user has selected the "non-commercial" registration option. 2) The user is not currently logged into the system.	
Main flow: Not required	
Extension use cases: None	
Inclusion use cases:	

1. ValidateEmail
Specialisation of: RegisterAccount
Postconditions: 1) A new member record is created with the email as the username. 2) An automated email is sent containing a 10-symbol random password. 3) The user is flagged for a mandatory password change on first login. 4) The user becomes eligible for a 10% discount on every 10th order.
Alternative flows: Email Already Registered

Use case ID: UC15	**Use case:** ValidateEmail
Brief Description: This use case verifies that the email address provided by an applicant (Non-commercial) is well-formed and follows standard formatting rules.	
Primary actors: IPOS-SA	
Secondary actors: None	
Preconditions: 1) The user has completed a registration form. 2) The IPOS-PU system is connected to the IPOS-SA API	
Main flow: Not required	
Extension use cases: None	
Inclusion use cases: None	
Postconditions: 1) Validation is successful, registration is confirmed. 2) For commercial customers, application is forwarded to IPOS-SA.	
Alternative flows: Invalid Format Validation failed	

Use case ID: UC16	**Use case:** ValidateDetails
Brief Description: This use case verifies the completeness and format of the extensive data required for a commercial membership application, including the company registration number, director details, and business address.	
Primary actors: IPOS-SA	
Secondary actors: None	
Preconditions: 1) The user has selected the "commercial" registration option. 2) The user has filled in the required business details in the application form.	
Main flow: Not required	
Extension use cases: None	
Inclusion use cases: None	

Postconditions: 1) The business data is confirmed as well-formed and ready for transmission to IPOS-SA.
Alternative flows: Incomplete Data Invalid Format Validation Failed

Use case ID: UC17	**Use case:** SMTP_Notification
Brief Description: This use case facilitates the delivery of automated email communications to customers and members. It acts as the execution layer for sending system alerts, such as order status updates, service requests from other subsystems, and registration credentials.	
Primary actors: SMTP	
Secondary actors: Registered Customer	
Preconditions: 1) The IPOS-PU-COMMS package is configured with valid SMTP server credentials. 2) The recipient has provided a valid email address.	
Main flow: Not required	
Extension use cases: None	
Inclusion use cases: None	
Postconditions: 1) The email is successfully handed off to the SMTP server. 2) If the service is unavailable, the details (timestamp, recipient, body) are recorded in a database table for later processing.	
Alternative flows: SMTP Connection Failure Invalid Recipient Address	

Use case ID: UC18	**Use case:** RequestSMTP_Service
Brief Description: This use case provides a dedicated communication interface for external subsystems to request the delivery of automated emails. It serves as the bridge between internal IPOS logic and the external SMTP infrastructure.	
Primary actors: IPOS-SA, IPOS-CA	
Secondary actors: None	
Preconditions: 1) A valid connection exists between the requesting subsystem (SA or CA) and the IPOS-PU interface. 2) The request contains all necessary email metadata, including the recipient's address, subject line, and message body.	
Main flow: Not required	
Extension use cases: None	
Inclusion use cases: 1. SMTP_Notification	
Postconditions: 1) The request is successfully passed to the SMTP_Notification use case for processing.	

2) A status confirmation (e.g., "Request Received") is returned to the calling subsystem.
Alternative flows: Invalid Request Data

Use case ID: UC19	**Use case:** Checkout
Brief Description: This use case facilitates the final stage of a transaction where the system calculates the final price, including VAT and applicable discounts, and processes the payment. It ensures that stock is only deducted from IPOS-CA after a successful payment is confirmed.	
Primary actors: Registered Customer, Unregistered User	
Secondary actors: Payment Processor	
Preconditions: 1) The customer has items in their shopping cart. 2) The system has a valid connection to the merchant's inventory on IPOS-CA.	
Main flow: Not required	
Extension use cases: 1. Check10thPurchaseDiscount	
Inclusion use cases: 1. MakePurchase 2. CalculateTax 3. CheckPromoDiscount 4. PaymentRequest 5. UpdateCart 6. CalculateTotal	
Postconditions: 1) On successful payment, stock is deducted from IPOS-CA. 2) Transaction and payment details are forwarded to IPOS-SA. 3) A confirmation email is sent via the SMTP_Notification use case.	
Alternative flows: Payment Failed Stock No Longer Available	

Use case ID: UC20	**Use case:** AddToCart
Brief Description: This use case allows a customer to select a specific quantity of a product and add it to their shopping cart.	
Primary actors: Unregistered User, Non-commercial Customer	
Secondary actors: None	
Preconditions: 1) The user is currently viewing a product's details. 2) The product is listed as "available" in the catalogue.	
Main flow: Not required	
***Extension use cases*:** None	
Inclusion use cases:	

1. UpdateCart
Postconditions: 1) The shopping cart is updated with the new item and quantity. 2) The cart's cumulative total and VAT are recalculated.
Alternative flows: Insufficient Stock

Use case ID: UC21	**Use case:** CalculateTotal
Brief Description: This use case calculates the cumulative cost of all items in the user's shopping cart. It factors in individual unit prices, applies any relevant promotional or member discounts, and calculates the applicable VAT for each item.	
Primary actors: Unregistered User, Non-commercial Customer	
Secondary actors: None	
Preconditions: 1) The user has at least one item in their virtual shopping cart. 2) The system has retrieved the current price and VAT status for each item from the merchant database.	
Main flow: Not required	
***Extension use cases*:** None	
Inclusion use cases: None	
Postconditions: 1) A subtotal, total discount amount, VAT amount, and final total are generated and displayed. 2) The calculated final total is passed to the PaymentProcessor.	
Alternative flows: None	

Use case ID: UC22	**Use case:** UpdateCart
Brief Description: This use case manages the state of the shopping cart by adding, removing, or adjusting the quantity of items. It ensures the session data is synchronised before totals are calculated or a payment request is sent.	
Primary actors: Unregistered User, Non-commercial Customer	
Secondary actors: None	
Preconditions: 1) The user has an active session on the IPOS-PU portal. 2) For additions, the system has confirmed the item exists in the merchant database.	
Main flow: Not required	
***Extension use cases*:** None	
Inclusion use cases: None	
Postconditions: 1) The shopping cart session data is updated to reflect the current item list and quantities. 2) The updated state is available for the CalculateTotal use case to perform pricing.	

Alternative flows: Remove Item

Use case ID: UC23	**Use case:** CalculateTax
Brief Description: This use case determines the amount of VAT applicable to each product in the shopping cart. It distinguishes between different categories of goods, such as prescription and non-prescription drugs, which may have different tax rates.	
Primary actors: Unregistered User, Non-commercial Customer	
Secondary actors: None	
Preconditions: 1) The customer has initiated the Checkout process. 2) The items in the cart have been retrieved with their associated tax category flags from the merchant database.	
Main flow: Not required	
***Extension use cases*:** None	
Inclusion use cases: None	
Postconditions: 1) The VAT amount for each line item is calculated. 2) The cumulative VAT total is added to the order's final total.	
Alternative flows: None	

Use case ID: UC24	**Use case:** TimeoutPromotion
Brief Description: This use case automatically monitors and deactivates promotional campaigns once their end date or time has passed. It ensures that the merchant catalogue and checkout calculations reflect current, valid pricing by updating the promotion status in the database.	
Primary actors: Time	
Secondary actors: None	
Preconditions: 1) The IPOS-PU system is operational and the scheduler is running. 2) Promotional data (start/end times) has been loaded into the system from the merchant database.	
Main flow: Not required	
***Extension use cases*:** None	
Inclusion use cases: None	
Postconditions: 1) Expired promotions are marked as "Inactive." 2) The product catalogue immediately reverts to standard pricing for the affected items. 3) Active user sessions are notified if a promotion expires while they are browsing.	

Alternative flows: None

Use case ID: UC25	**Use case:** Check10thPurchaseDiscount
Brief Description: This use case verifies if a logged-in non-commercial member is eligible for a 10% loyalty discount. Eligibility is determined by checking if the current order is the user's 10th purchase (or a multiple thereof).	
Primary actors: Non-commercial Customer	
Secondary actors: None	
Preconditions: 1) The user is logged in as a Non-Commercial Member. 2) The user has initiated the Checkout process. 3) The system has retrieved the user's order history from the database.	
Main flow: Not required	
***Extension use cases*:** None	
Inclusion use cases: None	
Specialisation Of: CheckDiscount	
Postconditions: 1) If the user is eligible, a 10% discount is applied to the order total. 2) The discount is compounded with any other active promotional discounts.	
Alternative flows: None	

Use case ID: UC26	**Use case:** CheckDiscount
Brief Description: This general use case defines the logic for identifying, validating, and applying price reductions to a customer's order.	
Primary actors: Unregistered User, Non-commercial Customer	
Secondary actors: None	
Preconditions: 1) The user has initiated the checkout process or is viewing their cart. 2) The system has access to the current product prices and the user's order history.	
Main flow: Not required	
***Extension use cases*:** None	
Inclusion use cases: None	
Postconditions: 1) The applicable discount amount is subtracted from the subtotal. 2) The "Discount Total" is updated for display to the user.	
Alternative flows: Compounding discounts	

Use case ID: UC27	**Use case:** CheckPromoDiscount
Brief Description: This specialized use case identifies and applies discounts associated with specific advertising campaigns. It compares the current system time against the campaign's active window and applies the designated reduction to eligible items in the cart.	
Primary actors: Unregistered User, Non-commercial Customer	
Secondary actors: None	
Preconditions: 1) The user has initiated the Checkout process with items in their cart. 2) The system has retrieved active promotional data (start/end dates and discount percentages) from the merchant database.	
Main flow: Not required	
***Extension use cases*:** None	
Inclusion use cases: None	
Specialisation Of: CheckDiscount	
Postconditions: 1) Any valid promotional discounts are subtracted from the item subtotals. 2) The system is ready to compound this with the Check10thPurchaseDiscount if the user is a non-commercial member.	
Alternative flows: Promotion expired No eligible items	

Use case ID: UC28	**Use case:** PaymentRequest
Brief Description: This use case initiates the external payment authorization process. It sends the final calculated order total to the Payment Processor and, upon receipt of a successful authorisation code, triggers the internal logging of the financial event.	
Primary actors: PaymentProcessor	
Secondary actors: None	
Preconditions: 1) The CalculateTotal and CalculateTax use cases have been completed. 2) The customer has provided valid payment method details. 3) A secure connection to the Payment Processor's API is established.	
Main flow: Not required	
***Extension use cases*:** None	
Inclusion use cases: 1. RecordTransaction	
Postconditions: 1) A "Success" or "Failure" status is returned to the Checkout flow. 2) The transaction details are stored in the IPOS-PU database via the RecordTransaction use case. 3) If successful, the system proceeds to deduct stock from IPOS-CA.	
Alternative flows:	

Payment declined Communication timeout

Use case ID: UC29	**Use case:** RecordTransaction
Brief Description: This use case creates a permanent record of a financial transaction. It captures the outcome of a payment attempt (Success/Failure), associates it with a specific Order ID, and ensures the sales data is updated for both local records and the central IPOS-SA system.	
Primary actors: PaymentProcessor, IPOS-SA	
Secondary actors: Unregistered User, Non-commercial Customer	
Preconditions: 1) A PaymentRequest has been initiated. 2) The Payment Processor has returned a transaction status (e.g., Authorized, Declined, or Error).	
Main flow: Not required	
***Extension use cases*:** None	
Inclusion use cases: None	
Postconditions: 1) A new transaction record is created in the IPOS-PU database. 2) The corresponding Order status is updated (e.g., from "Pending" to "Paid"). 3) Transaction metadata is queued or transmitted to IPOS-SA for financial reporting.	
Alternative flows: None	

Use case ID: UC30	**Use case:** QueryPromotion
Brief Description: This use case retrieves active promotional data for a specific product or set of products. It verifies if the current system time falls within the promotion's validity window and returns the applicable discount percentage.	
Primary actors: Unregistered User, Non-commercial Customer	
Secondary actors: None	
Preconditions: 1) The product ID(s) being queried are valid and exist in the catalogue. 2) The promotional database is accessible.	
Main flow: Not required	
***Extension use cases*:** None	
Inclusion use cases: None	
Postconditions: 1) The system returns the current applicable discount rate (e.g., 15%). 2) If no active campaign is found, the system returns a rate of 0%.	
Alternative flows: Expired promotion	

Use case ID: UC31	**Use case:** MakePromotion

Brief Description: This use case allows an authorized administrator to define a new advertising campaign. It involves setting the discount percentage, identifying the target products, and specifying the start and end dates. The system then communicates this request to the client application to ensure the promotion is synchronised.
Primary actors: Admin
Secondary actors: Commercial Customer
Preconditions: 1) The Staff member is successfully logged in with administrative privileges. 2) The target products are currently active and available in the merchant's catalogue.
Main flow: Not required
***Extension use cases*:** None
Inclusion use cases: 1. RequestPromotion
Postconditions: 1) A new promotional record is created and set to "Pending" or "Active" based on the start date. 2) The IPOS-CA subsystem is notified of the new campaign parameters.
Alternative flows: Invalid date range

Use case ID: UC32	**Use case:** RequestPromotion
Brief Description: This use case handles the transmission of promotional data from the online portal to the merchant's client application. It packages the campaign details and sends them via the subsystem interface to ensure local inventory records match the portal's marketing.	
Primary actors: Commercial Customer	
Secondary actors: None	
Preconditions: 1) The administrator has completed the definition of a new promotion in the MakePromotion use case. 2) The IPOS-PU system has established a valid network connection with the targeted IPOS-CA terminal.	
Main flow: Not required	
***Extension use cases*:** None	
Inclusion use cases: None	
Postconditions: 1) The promotional data is successfully delivered to the IPOS-CA subsystem. 2) A receipt confirmation is logged in the IPOS-PU system to verify synchronisation.	
Alternative flows: None	

Use case ID: UC33	**Use case:** CancelPromotion
Brief Description: This use case allows an authorised Staff member to manually deactivate an active or pending promotional campaign. The system updates the promotion status and ensures that the discount is no longer applied during checkout or displayed in the catalogue.	
Primary actors: Admin	

Secondary actors: None
Preconditions: 1) The Staff member is successfully logged into the IPOS-PU portal with administrative privileges. 2) At least one promotional campaign (Pending or Active) exists in the system.
Main flow: Not required
***Extension use cases*:** None
Inclusion use cases: 1. SearchPromotion
Postconditions: 1) The selected promotion is marked as "Inactive/Cancelled" in the database. 2) The product catalogue is updated to reflect standard pricing for the affected items. 3) A log entry of the cancellation (including the Staff ID and timestamp) is created for audit purposes.
Alternative flows: Promotion already expired Checkout in-progress

Use case ID: UC34	**Use case:** SearchPromotion
Brief Description: This use case allows an authorised Staff member to locate specific promotional campaigns within the system using various filters such as campaign name, date range, or associated product IDs.	
Primary actors: Admin	
Secondary actors: None	
Preconditions: 1) The Staff member is authenticated and has the necessary permissions to view promotional data. 2) The promotional database is accessible.	
Main flow: Not required	
***Extension use cases*:** None	
Inclusion use cases: None	
Postconditions: 1) The system presents a list of promotional campaigns that match the search criteria. 2) The Staff member can select a specific promotion for further action (e.g., viewing details or cancellation).	
Alternative flows: No results found	

Use case ID: UC35	**Use case:** GenerateReport
Brief Description: This general use case defines the shared logic for creating data summaries within the IPOS-PU system. It handles common tasks such as setting date ranges, selecting output formats (e.g., CSV, PDF), and triggering the data retrieval process.	
Primary actors: Admin	
Secondary actors: None	
Preconditions:	

1) The Staff member is successfully logged into the IPOS-PU administrative portal. 2) The relevant data stores (Sales, Promotion, or Traffic logs) are online and accessible.
Main flow: Not required
***Extension use cases*:** None
Inclusion use cases: None
Postconditions: 1) A structured report is generated and displayed to the user or downloaded to their local machine. 2) The report generation event is logged in the system audit trail.
Alternative flows: No data in range

Use case ID: UC36	**Use case:** GenerateClickReport
Brief Description: This specialised use case analyses user interaction data to identify which products, advertisements, or promotional links received the most engagement ("clicks"). It is used by Staff to measure the effectiveness of the portal's marketing layout.	
Primary actors: Admin	
Secondary actors: None	
Preconditions: 1) The Staff member is authenticated and has access to the reporting dashboard. 2) The system has been logging clickstream data (user interactions) in the portal's traffic database.	
Main flow: Not required	
***Extension use cases*:** None	
Inclusion use cases: None	
Specialisation Of: GenerateReport	
Postconditions: 1) The system generates a report detailing click counts per item/promotion within the specified timeframe. 2) The report highlights the "top-performing" links to assist with future advertising placement.	
Alternative flows: None	

Use case ID: UC37	**Use case:** GeneratePromotionReport
Brief Description: This specialised use case generates a summary of all promotional activities within a defined period. It evaluates the performance of advertising campaigns by calculating total discounts granted and comparing them against standard revenue to determine marketing ROI (Return on Investment).	
Primary actors: Staff	
Secondary actors: None	
Preconditions: 1) The Staff member is authenticated and has navigated to the Reporting module. 2) The promotional database contains historical or active campaign data.	
Main flow: Not required	

***Extension use cases*:** None
Inclusion use cases: None
Specialisation Of: GenerateReport
Postconditions: 1) A detailed report is generated showing campaign effectiveness, total value of discounts applied, and product performance under promotion.
Alternative flows: None

Use case ID: UC38	**Use case:** GenerateSalesReport
Brief Description: This specialised use case extracts and summarizes financial transaction data from the portal. It calculates total revenue, net amounts, and VAT collected over a specific period.	
Primary actors: Admin	
Secondary actors: IPOS-SA	
Preconditions: 1) The Staff member is authenticated and has administrative permissions. 2) The sales database service is operational.	
Main flow: Not required	
***Extension use cases*:** None	
Inclusion use cases: None	
Specialisation Of: GenerateReport	
Postconditions: 1) A comprehensive sales report is generated, categorized by date, merchant, or product type. 2) The report data is formatted for export.	
Alternative flows: None	

Use case ID: UC39	**Use case:** RequestPaymentService
Brief Description: This use case manages the interaction with the Payment Processor. It securely transmits the encrypted payment details and the final order total, then awaits a synchronous response (Approval/Decline) to allow the checkout process to continue.	
Primary actors: PaymentProcessor, IPOS-CA, IPOS-SA	
Secondary actors: None	
Preconditions: 1) The user has reached the final stage of Checkout. 2) The CalculateTotal and CalculateTax use cases have provided a final, non-zero amount. 3) A secure HTTPS connection is established with the Payment Processor.	
Main flow: Not required	
***Extension use cases*:** None	
Inclusion use cases: None	

Postconditions: 1) The system receives a unique Transaction ID and an authorization status from the processor. 2) The status is passed back to the portal to determine if the order should be finalized or if the user should be prompted for a new payment method.
Alternative flows: None

Use case ID: UC40	**Use case:** UpdateOrderStatus
Brief Description: This use case updates the current state of a customer's order in the database. When a status change occurs (e.g., payment confirmed, order shipped, or order cancelled), the system triggers an automated notification to the user to maintain transparency.	
Primary actors: IPOS-CA	
Secondary actors: None	
Preconditions: 1) A valid Order ID exists in the system. 2) The actor has the authority to change the status (e.g., the Payment Processor has confirmed funds, or a Staff member has marked the item as shipped).	
Main flow: Not required	
***Extension use cases*:** None	
Inclusion use cases: 1. SMTP_Notification	
Postconditions: 1) The order's status is updated in the database. 2) An audit log entry is created recording the change, the timestamp, and the actor responsible. 3) A notification is successfully queued in the SMTP server.	
Alternative flows: None	

4.2.1.3 Prioritised list of use cases

The list below prioritises the use cases included in the use case model according to their importance for the IPOS-PU subsystem. InfoPharma's development team lead will use the assigned priorities to decide on the order of implementing the functionality specified in the use cases according to the resources available: the ***essential*** use cases will be implemented first, then the use cases ranked as ***important*** will follow. The use cases ranked as ***good-to-have*** will be implemented last provided, the resources (manpower and time) are available to complete their implementation.

Use case ID	Use case name	UC priority	Justification
UC01	MakePurchase	Essential	Allows customers to purchase products through the system.

UC02	ViewItem	Essential	This functionality allows users to view detailed product information. It is essential for informed purchasing decisions.
UC03	ViewCatalogue	Essential	This is essential as users must browse available products and services before any transaction can occur.
UC04	SearchCatalogue	Important	This functionality helps customers find products more easily. The system can still operate without it as customers are able to browse the catalogue.
UC05	Login	Important	Important as it allows registered users to access accounts and stored information securely.
UC06	Logout	Good to have	Improves session security but system can still operate without explicit logout.
UC07	CustomerAuthentication	Important	This is important for system security. It ensures secure access to protected account features.
UC08	PromptPasswordChange	Good to have	Enhances long-term account security but not required for core system operations.
UC09	ChangePassword	Good to have	Security enhancement allowing users to maintain account protection.
UC10	EditUserDetails	Good to have	Improves usability by allowing profile updates but not critical for purchases.
UC11	ViewCart	Essential	Users must review selected items before proceeding to checkout. This is essential for verifying orders before checkout.

UC12	RegisterAccount	Important	This functionality allows commercial and non-commercial users to create an account. This is essential for authentication, order tracking and accessing member specific features.
UC13	RegisterCommercialAccount	Important	Supports business customers, expanding system usage but not blocking core retail use.
UC14	RegisterNonCommercialAccount	Important	Enables standard customer onboarding for account-based purchases.
UC15	ValidateEmail	Important	This functionality is important as it ensures the email address provided is valid before creating the account.
UC16	ValidateDetails	Important	Important as it ensures commercial user details are correct before approval.
UC17	SMTP_Notification	Important	Provides transaction confirmations and user communications improving reliability.
UC18	RequestSMTP_Service	Important	This is important as it allows the system to send emails for registration and transaction notifications. This ensures users receive important system communications.
UC19	Checkout	Essential	This is essential as it is required to finalise purchases and confirm order submission.
UC20	AddToCart	Essential	This is essential as it allows users to collect selected items before proceeding to checkout.
UC21	CalculateTotal	Essential	This is an essential part of the checkout process. Before the payment can be completed, the

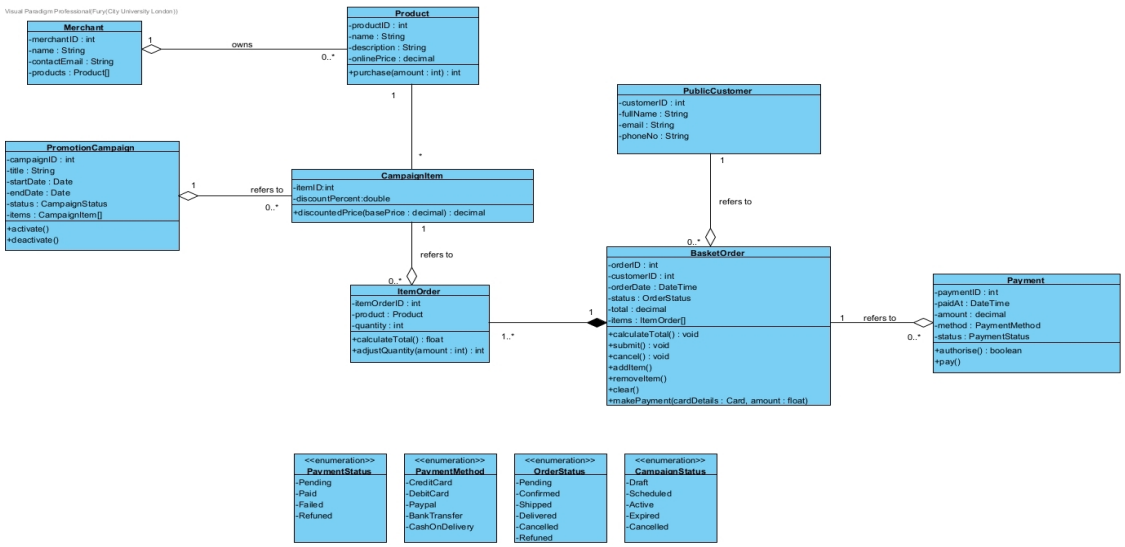
			system must calculate the total.
UC22	UpdateCart	Essential	Allows users to modify quantities or remove items before checkout to ensure order accuracy.
UC23	CalculateTax	Good to have	Needed for pricing accuracy but system purchasing logic can still function conceptually without it.
UC24	TimeoutPromotion	Good to have	Handles promotion expiry but Admin can manually manage promotions so this just helps with efficiency.
UC25	Check10thPurchaseDiscount	Important	Provides reward to non-commercial users which improves customer engagement but is not essential.
UC26	CheckDiscount	Essential	This functionality is essential to make sure customers are applying the right discount when purchasing products.
UC27	CheckPromoDiscount	Essential	Essential to ensure that promotions are accurately applied during checkout.
UC28	PaymentRequest	Essential	Essential as it sends payment information to the payment service to complete the purchase.
UC29	RecordTransaction	Essential	This functionality is essential as it records completed transactions and ensures payments are properly tracked.
UC30	QueryPromotion	Important	This functionality ensures correct promotion eligibility during purchases.
UC31	MakePromotion	Important	This functionality is important because the system can operate without promotions. However, promotions increase business value.
UC32	RequestPromotion	Important	This functionality supports the promotion workflow by allowing promotions to be requested and managed.

UC33	CancelPromotion	Important	This feature allows Admin to manage promotions more efficiently but can go without constant updating.
UC34	SearchPromotion	Good to have	This helps Admin to find promotions easily but is not essential for promotion to function.
UC35	GenerateReport	Good to have	Reports can help Admin to review system activity and performance but not required for the system to operate.
UC36	GenerateClickReport	Good to have	This functionality generates reports based on user interactions. It is useful for analysing user behaviour, but it is not required for system functionality.
UC37	GeneratePromotionReport	Good to have	This functionality generates reports that are useful for promotion analysis.
UC38	GenerateSalesReport	Good to have	It is useful for monitoring performance and helps with business insights. Not necessary for system functionality.
UC39	RequestPaymentService	Essential	This is essential as it allows the system to process payments and receive the payment outcome.
UC40	UpdateOrderStatus	Important	This is important as it updates the order status after payment. This provides accurate order tracking for users.

4.2.1.4 Class diagram (simplified)

This model is solely to be referenced and used by internal team members for the initial phases of development of the IPOS software.

Though the detailed design and implementation of IPOS-PU will take place later, this simplified class diagram begins to depict the classes, interfaces, and enumerations expected in the detailed model.



5 Anticipated evolution of system requirements

The InfoPharma IPOS-PU subsystem will be deployed as part of the company's switch from the current manual system to a digitalised platform, however, it does not completely replace the paper-based system that precedes it. Feedback and information gathered from the users, merchants and the stakeholders will shape future changes, and the expected evolution is summarised below in a non-exhaustive list.

5.1 Integration with other IPOS subsystems

Currently, systems operate without real-time integration between inventory, catalogue management, and ordering processes.

As of now systems operate without any synchronisation between the inventory, the catalogue and the ordering processes – this prevents the implementation of locks which could otherwise be used to improve data integrity. To counter this, we may implement a centralised database, shared across all subsystems, with a specialised interface granting functions for each subsystem to enforce the Principle of Least Privilege (PoLP).

5.2 User interface and customer experience

Currently, customers interact through direct commutation methods such as through email or phone, which isn't convenient for consumers. We plan on changing this by introducing an intuitive user interface for notifications that can be accessed within the application itself.

5.3 Promotional features and merchant interaction

There aren't any promotional activities limited and not dynamically managed through the digital system. So we vision that merchants can manage promotional campaigns through the promotional content and be able to apply any discounts.

6 IPOS high-level design

This section defines the high-level design of the entire IPOS software, which includes its architecture – the subsystems which it is made up of, and the communication interfacts between the subsystems.

6.1 Subsystems

The subsystems envisaged are:

6.1.1 IPOS-SA

This subsystem will allow Retail Managers of InfoPharma to manage the electronic catalogue of pharmaceutical products provided to merchants, provide order forms, control availability of stock, and generate detailed reports using the system database.

6.1.2 IPOS-CA

This subsystem will allow Merchants to access IPOS-SA's order forms through an interface and track the status of such orders, manage account holders, track local stock availability and automate orders, record sales, and generate reports.

6.1.3 IPOS-PU

This subsystem will allow members of the public to purchase pharmaceutical goods from merchants, access promotions, and register with InfoPharma to receive special discounts. The subsystem will also allow Online Merchants to register with IPOS-SA, and reports may be generated by Admins.

6.2 Software architecture

The architecture of IPOS as a UML component diagram, developed with Visual Paradigm, is shown on ***Figure 2***.

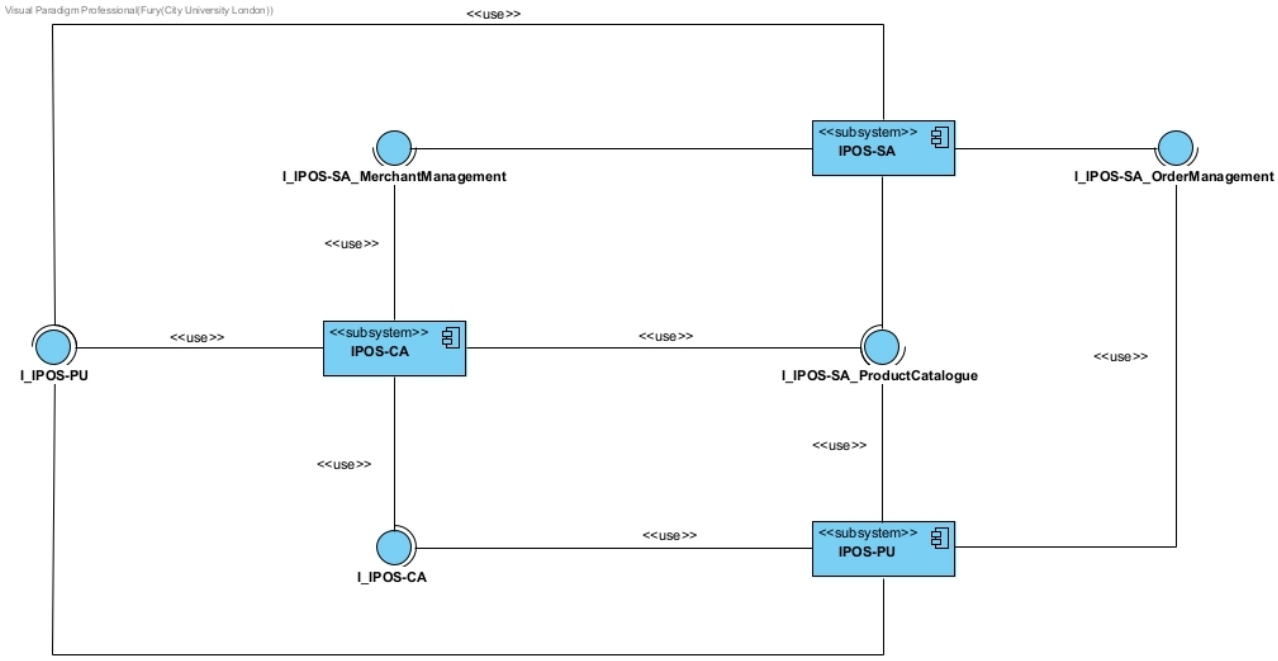


Figure 2*: IPOS architecture as a UML component diagram. Each subsystem is modelled as a component with stereotype <<subsystem>>. The communication between the subsystems is achieved via a set of assembly connectors (i.e., a pair of "provided" and "required" interfaces referring to the API offered by the respective subsystems. I_IPOS-PU and I_IPOS-CA are provided by IPOS-PU and IPOS-CA respectively, and the APIs are used to access the functionality offered by one of the subsystems by the other two subsystems. IPOS-SA specifically provides three interfaces, separating the APIs used for managing merchants, accessing/updating the product catalogue, and managing orders - these are represented by I_IPOS-SA_MerchantManagement, I_IPOS-SA_ProductCatalogue, and I_IPOS-SA_OrderManagement respectively. Whilst I_IPOS-SA_ProductCatalogue and I_IPOS-SA_OrderManagement are provided to both subsystems, I_IPOS-SA_MerchantManagement is solely provided to the IPOS-CA subsystem.*

Figure 2 hides the API details. Using the “presentation options” of Visual Paradigm, one can switch to the “class” notations and disclose full interface details, as is illustrated on ***Figure 3***.

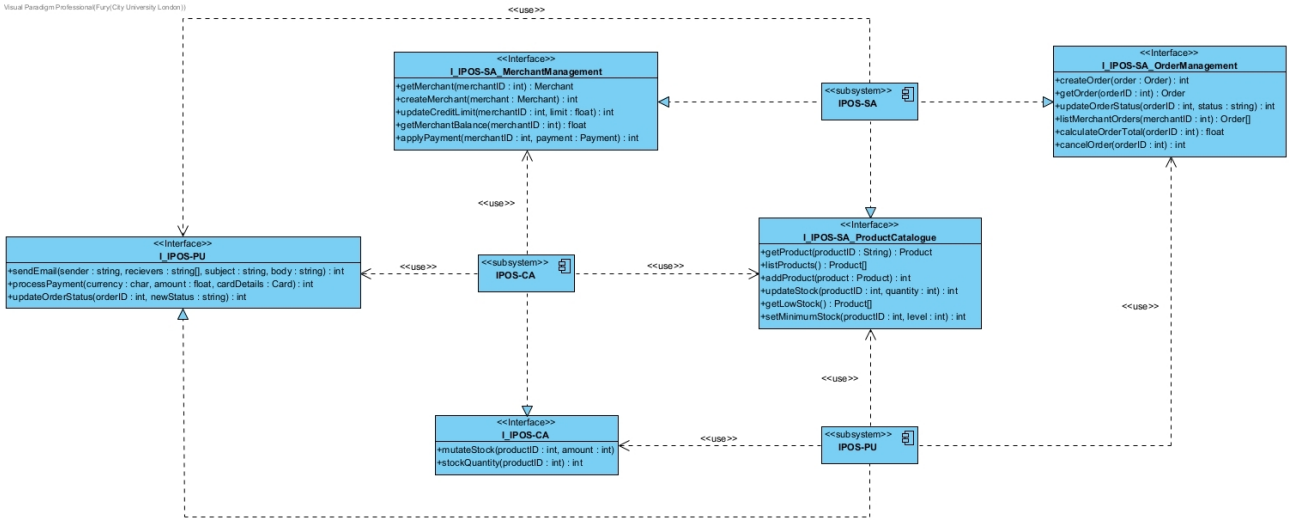


Figure 3*: Now the methods defined in interfaces are explicitly shown with their full signatures. These are plausible public methods via which the functionality of respective subsystems can be accessed, per collaborative discussions with the teams responsible for the high-level designs of IPOS-SA and IPOS-CA respectively as well. Note that with this change of presentation of the interfaces, the relationships have changed to be more specific as well: the subsystems responsible to implement the interface has a "realisation" relationship with it, implying respective subsystems will contain implementation classes realising the interface. Subsystems that will use an interface are depicted with a usage relationship pointing to the interface that will be used, this can be seen through the "<<use>>" label.*

The IPOS architecture includes three subsystems and shows the five necessary interfaces for these subsystems to interact with each other effectively.

6.3 Design diagram of subsystem IPOS-PU

Next, we elaborate the high-level design of IPOS-PU and provide details on the interfaces that this subsystem requires, and on the implementation of the subsystem's own required interface that it provides.

The class diagram of the IPOS-PU subsystem is shown in ***Figure 4***.

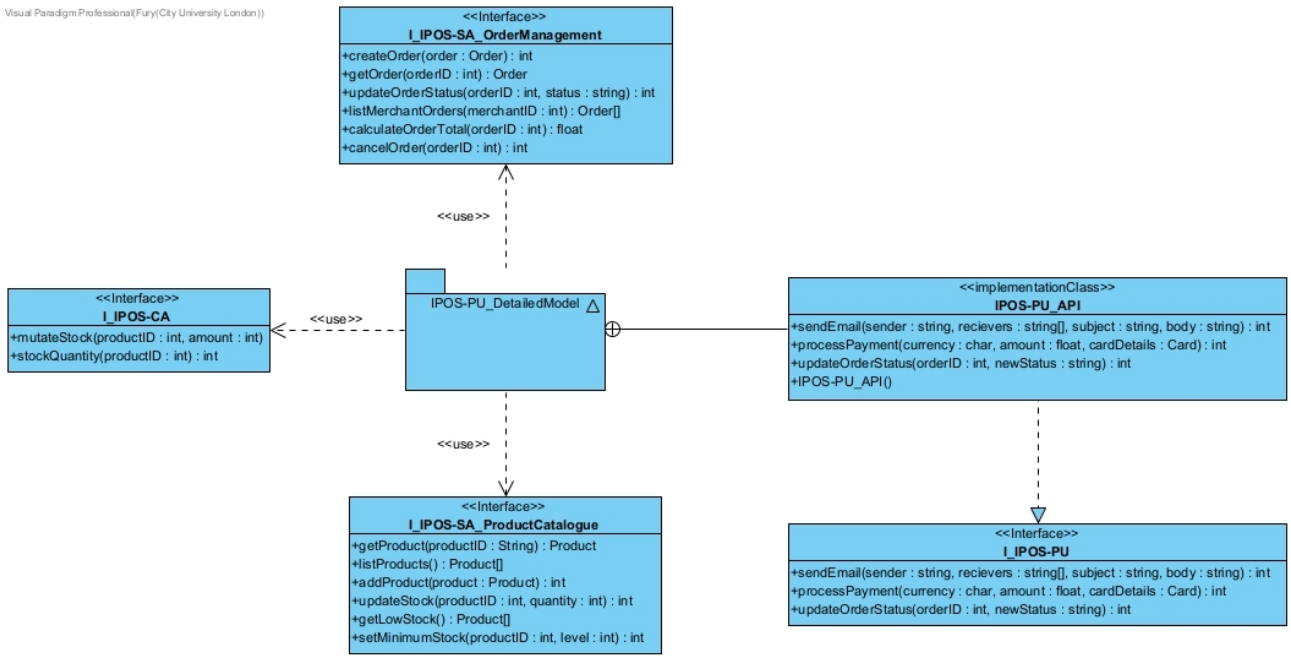


***Figure 4**: High-level design of IPOS-PU - this diagram shows the interfaces that IPOS-PU has dependencies on (which already appeared in full detail in **Figure 3**), and a minimal version of the implementation class I_IPOS-PU_API of the interface I_IPOS-PU, together with the realisation relationship between the provided interface and the implementation class.*

Due to the complexities of interfacing with an SMTP server and payment processing API, the implementation class has been left minimal until these detailed design decisions have been made in the later stages of the project's development.

6.4 Source code (Java) for the interfaces of IPOS-PU

Now that we have defined all interfaces and the implementation class IPOS-PU_API, source code could be derived (e.g. Java source code) for the interfaces and the implementation classes shown in Figure 4. The complete Java code of the interfaces and a skeleton of the implementation class are shown below generated by Visual Paradigm is provided below, alongside clarifying comments to explain purpose and functionality:

```
public class IPOS_PU_API implements I_IPOS_PU {

      /**
      * Sends an email notification (e.g., order confirmation, receipt, status
      update).
      * This operation is expected to pass the email details to an email service
      (or simulated service).
      *
      * @param sender the email address that the message appears to come from.
      * Constraint: should be a non-empty, valid email format address.
      * @param recievers the list of recipient email addresses.
      * Constraint: array should not be null/empty; each entry should be a valid
      email format.
      * @param subject the email subject line.
      * Constraint: should be non-null; typically short (may be limited by email
      service).
      * @param body the main message content.
```

```
	* Constraint: should be non-null; may have a maximum length depending on
	service.
	*/

	public int sendEmail(String sender, String[] recievers, String subject,
	String body) {
		throw new UnsupportedOperationException();
	}

	/**
	* Processes an online card payment for an order.
	* This operation is expected to contact a payment service (or simulated
	service) and return a status code.
	*
	* @param currency the currency used for the payment.
	* Constraint: must be a supported currency indicator used by the
	subsystem/payment simulator.
	* @param amount the amount to charge.
	* Constraint: must be greater than 0; should represent the calculated
	order total.
	* @param cardDetails card information required to attempt authorisation.
	* Constraint: must not be null; card number/expiry/security code must be
	present and valid format.
	*/

	public int processPayment(char currency, float amount, Card cardDetails) {
		throw new UnsupportedOperationException();
	}

	/**
	* Updates the status of an existing online order.
	* This operation is expected to persist the new status in the order
	datastore and/or notify other subsystems.
	*
	* @param orderID the identifier of the order to update.
	* Constraint: must refer to an existing order in the datastore.
	* @param newStatus the new status value for the order.
	* Constraint: should be a valid status recognised by the system workflow
	(e.g. Pending/Confirmed/Shipped).
	*/

	public int updateOrderStatus(int orderID, String newStatus) {
		throw new UnsupportedOperationException();
	}

	/**
	* Default constructor
	*/

	public IPOS_PU_API() {
		throw new UnsupportedOperationException();
	}
}

--------------------------------------------------------------------------------

public interface I_IPOS_SA_OrderManagement {

	/**
	* Creates a new order record in the Sales Administration (SA) subsystem.
	* This is expected to store the order and allocate an order ID (or return
```

```
a status code).
*
* @param order the order object containing customer, items, totals, and
initial status.
* Constraint: must not be null; must contain at least one order item;
totals should be consistent with items.
*/

public int createOrder(Order order);

/**
* Retrieves a previously created order from SA by its identifier.
*
* @param orderID the unique order identifier to search for.
* Constraint: must be a valid existing order ID; if not found, the
subsystem may return null.
*/

public Order getOrder(int orderID);

/**
* Updates an order's status within SA (e.g., moving from Pending to
Confirmed/Cancelled).
* Used to reflect progress of the order lifecycle.
*
* @param orderID the identifier of the order to update.
* Constraint: must refer to an existing order.
* @param status the new status to apply.
* Constraint: must be a valid workflow status; invalid transitions may be
rejected by the subsystem.
*/

public int updateOrderStatus(int orderID, String status);

/**
* Lists all orders belonging to a particular merchant.
* Used for merchant order history and management screens.
*
* @param merchantID the identifier of the merchant.
* Constraint: must refer to an existing merchant; if merchant has no
orders, may return empty array.
*/

public Order[] listMerchantOrders(int merchantID);

/**
* Calculates the current total cost of an order based on order lines,
quantities, and prices.
* Used to validate totals before payment and for reporting.
*
* @param orderID the identifier of the order to total.
* Constraint: must refer to an existing order with at least one item.
*/

public float calculateOrderTotal(int orderID);

/**
* Cancels an order in SA.
* This should only succeed if the order is in a cancellable state (e.g.,
not already shipped/delivered).
*
```

```
      * @param orderID the identifier of the order to cancel.
      * Constraint: must exist; cancellation may be rejected if state transition
      is not allowed.
      */

      public int cancelOrder(int orderID);
}
```

```
public interface I_IPOS_SA_ProductCatalogue {

        /**
       * Retrieves a product from the Sales Administration (SA) catalogue.
       *
       * @param productID the unique identifier of the product.
       * Constraint: must not be null or empty; must refer to an existing
      product in the datastore.
       */

      public Product getProduct(String productID);
      /**
       * Lists all products currently available in the catalogue.
       *
       * Constraint: may return an empty array if no products are available.
       */

      public Product[] listProducts();
      /**
       * Adds a new product to the catalogue.
       *
       * @param product the product object to add.
       * Constraint: must not be null; product ID must be unique; price should
      be >= 0.
       */

      public int addProduct(Product product);
      /**
       * Updates the stock level for a given product.
       *
       * @param productID the identifier of the product to update.
       * Constraint: must refer to an existing product.
       * @param quantity the new stock quantity.
       * Constraint: must be >= 0 (stock cannot be negative).
       */

      public int updateStock(int productID, int quantity);
      /**
       * Retrieves products whose stock level is below the defined minimum
      threshold.
       *
       * Constraint: threshold values must be defined for products in the
      system.
       */

      public Product[] getLowStock();
      /**
       * Sets the minimum stock threshold for a product.
       *
       * @param productID the identifier of the product.
       * Constraint: must refer to an existing product.
```

```
       * @param level the minimum stock level.
       * Constraint: must be >= 0.
       */

      public int setMinimumStock(int productID, int level);
      /**
       * Sends an email notification via the IPOS-PU subsystem.
       *
       * @param sender the email address of the sender.
       * Constraint: must be a valid email format.
       * @param receivers the recipient email addresses.
       * Constraint: must not be null or empty.
       * @param subject the email subject line.
       * Constraint: must not be null.
       * @param body the content of the email.
       * Constraint: must not be null.
       */
}
```

```
public interface I_IPOS_PU {

      public int sendEmail(String sender, String[] receivers, String subject,
      String body);
      /**
       * Processes a payment request through the payment service.
       *
       * @param currency the currency indicator used for the transaction.
       * Constraint: must be a supported currency.
       * @param amount the amount to charge.
       * Constraint: must be greater than 0.
       * @param cardDetails the card information required for authorisation.
       * Constraint: must not be null; must contain valid card information.
       */

      public int processPayment(char currency, float amount, Card cardDetails);
      /**
       * Updates the status of an order in the PU subsystem.
       *
       * @param orderID the identifier of the order.
       * Constraint: must refer to an existing order.
       * @param newStatus the new status value.
       * Constraint: must be a valid workflow status.
       */

      public int updateOrderStatus(int orderID, String newStatus);
}
```

```
public interface I_IPOS_CA {

      /**
       * Adjusts the stock level of a product.
       * Used when orders are completed or cancelled.
       *
       * @param productID the identifier of the product.
       * Constraint: must refer to an existing product.
       * @param amount the change in stock quantity.
       * Constraint: resulting stock must not fall below zero.
```

```
     */
    public void mutateStock(int productID, int amount);

    /**
     * Retrieves the current stock quantity of a product.
     *
     * @param productID the identifier of the product.
     * Constraint: must refer to an existing product.
     */
    public int stockQuantity(int productID);
}
```

The comments throughout this code are written with the intent of clarifying the purpose of both methods and parameters, as well as values that are returned – the project managers of subsystems IPOS-SA, IPOS-CA, and IPOS-PU respectively have all agreed on a REST API framework, meaning that methods will all return, at a bare minimum, an integer that represents a response to any request made through an interface.

6.5 Testing the communications between the subsystems

Now that we have defined the interfaces necessary for communication between the subsystems of IPOS, and generated Java code for all of them, we develop tests for inter-subsystem communication, i.e. of the respective implementation classes that IPOS-PU communicates with.

We have not yet committed to a particular testing platform to run the tests, and so the tests sketched for the methods of these implementation classes will be written generically.

6.5.1 IPOS-PU

Method	Test ID	Parameter values	Comment	Expected outcome
sendEmail(sender:string, recievers:string[], subject:string, body:string): int				
sendEmail(sender:string, recievers:string[], subject:string, body:string): int	1	sender = "abc@gmail.com" recievers = "def@gmail.com, ghi@gmail.com" subject = "RE: Email" body = "This is an email."	Emails of sender and recievers must exist.	Status code: 200 ["Success"] Email is sent from appropriate sender (abc), to appropriate recievers (def, ghi), with correct subject heading and body text.
sendEmail(sender:string, recievers:string[], subject:string, body:string): int	2	sender = "invalid_email"	Email is of invalid format.	Status code: 400 ["Failure"] No email is sent.
sendEmail(sender:string, recievers:string[], subject:string, body:string): int	3	recievers = ""	Email is empty.	Status code: 400 ["Failure"] No email is sent.
sendEmail(sender:string, recievers:string[], subject:string, body:string): int	4	recievers = "def@gmail.com, ghi@gmail.com, nothing@gmail.com"	Reciever email(s) do not exist.	Status code: 502 ["Failure"] The email is attempted to be sent, but will

				not send to the invalid address as the response from the SMTP gateway is invalid.
processPayment(currency:char, amount:float, cardDetails:Card): int				

processPayment(currency:char, amount:float, cardDetails:Card): int	5	currency = "£"
		amount = "10"
		cardDetails = {
		senderCardHolderName: "name"
		senderCardNo: "1234 5678 9012 3456"
		senderExpirationDate: "05-21"
		senderCVC: "123"
		recieverCardHolderName: "name2"
		recieverAccountNo: "1234 5678"
		recieverSortCode: "123 456"
		}
			Card details must correspond to a valid card with a sufficient amount of funds and a valid account able to receive funds.	Status code: 200 ["Success"]
				
				Payment of appropriate amount is made from sender account to reciever account.

processPayment(currency:char, amount:float, cardDetails:Card): int	6	senderCardHolderName: "" recieverAccountNo: ""	Card details of sender or reciever are incomplete.	Status code: 401 ["Failure"] No payment is made, and an authorization request is made to the client to complete the transaction.
processPayment(currency:char, amount:float, cardDetails:Card): int	7	currency = "GBP"	Currency is not supported format.	Status code: 400 ["Failure"] No payment is made.
processPayment(currency:char, amount:float, cardDetails:Card): int	8	amount = 10000	Funds are not available in sender account.	Status code: 502 No payment is made as the response from the Payment Processor gateway is invalid.
processPayment(currency:char, amount:float, cardDetails:Card): int	9		Too many invalid requests are made in a short period of time.	Status code: 429 The request is ignored and further requests are also ignored for a period of time ("rate limiting").
updateOrderStatus(orderID:int, newStatus:string): int				
updateOrderStatus(orderID: int, newStatus:string): int	10	orderID = 1 newStatus = "Shipped"	Order ID exists. Status is supported in enumeration class of statuses for	Status code: 200 ["Success"] Order status is updated.

			orders.	
updateOrderStatus(orderID: int, newStatus:string): int	11	orderID = one	Empty	Status code: 400 ["Failure"] Order status is not updated.
updateOrderStatus(orderID: int, newStatus:string): int	12	orderID = -1	Order ID does not exist.	Status code: 400 ["Failure"] Order status is not updated.
updateOrderStatus(orderID: int, newStatus:string): int	13	newStatus = "nothing"	Status is not supported.	Status code: 400 ["Failure"] Order status is not updated.

The returned data of these methods are typically status codes that align with the REST API framework, the specific details of the status codes present in these cases can be found below:

200 OK	Indicates that the request has succeeded.
400 Bad Request	The server could not understand the request due to incorrect syntax.
401 Unauthorized	Indicates that the request requires user authentication information.
429 Too Many Requests	The user has sent too many requests in a given amount of time ("rate limiting").
502 Bad Gateway	The server got an invalid response while working as a gateway to get the response needed to handle the request.

Further test-cases have been provided by the teams responsible for IPOS-SA and IPOS-CA, regarding their own interfaces and the methods that they provide to IPOS-PU:

6.5.2 IPOS-CA

Method	Test ID	Parameter values	Comment	Expected Outcome
mutateStock(productID:int, stockAmount:int): int	1	productID = 1001 stockAmount = 5	Product exists. Increasing stock.	returns 0 ["success"] Stock level increased by 5.
mutateStock(productID:int, stockAmount:int): int	2	productID = 1001 stockAmount = -3	Product exists. Sufficient stock available.	returns 0 ["success"] Stock level reduced by 3.
mutateStock(productID:int, stockAmount:int): int	3	productID = 1001 stockAmount = -50	Product exists but insufficient stock.	returns 3 ["insufficient stock"] Stock unchanged.

mutateStock(productID:int, stockAmount:int): int	4	productID = 9999 stockAmount = 5	Product does not exist.	returns 1 [“product not found”]. No stock updated.
mutateStock(productID:int, stockAmount:int): int	5	productID = -1 stockAmount = 5	Invalid productID (negative value).	returns 4 [“invalid product ID”] Validation error. No update performed.
mutateStock(productID:int, stockAmount:int): int	6	productID = 1001 stockAmount = 0	Zero stock change requested.	returns 2 [“invalid stock amount”] No update performed.
mutateStock(productID:int, stockAmount:int): int	7	productID = 1001 stockAmount = 999999	Stock amount exceeds allowed limit.	returns 2 [“invalid stock amount”] No update performed.
mutateStock(productID:int, stockAmount:int): int	8	productID = 1001 stockAmount = -5	System/database error occurs during execution. The method does not return any standard business status codes (e.g., success, invalid input, insufficient stock).	Returns 9 [“system/unknown error”] No update performed. Error logged for human intervention.
withdrawStock(productID:int, stockAmount:int)	9	productID=101, stockAmount=5	Stock available ≥ 5	Stock reduced. Returns success.
withdrawStock(productID:int, stockAmount:int)	10	productID=101, stockAmount=50	Stock available < 50	Operation fails. Stock unchanged.
stockAvailability(productID:int)	11	productID=101	Product exists	Returns correct quantity.
stockAvailability(productID:int)	12	productID=999	Product not found	Returns 0 or error indicator.

6.5.3 IPOS-SA

IPOS-SA were unable to provide test-cases for their interfaces in the given time, as such, this section is left purposefully blank – this is due to their methods relying on ‘Order’ and ‘Product’ data types, which our team are unable to define, due to it being out of scope for IPOS-PU’s subsystem.

However, we have approximated simple test-cases in their stead for the methods IPOS-PU uses:

Method	Test ID	Parameter values	Comment	Expected outcome
getOrder(orderID: int): Order				
getOrder(orderID:int): Order	1	orderID = 1	ID must exist/be valid and correspond to an order.	Status code: 200 [“Success”] Order is returned.

getOrder(orderID:int): Order	2	orderID = 10000	ID does not exist or does not correspond to an order.	Status code: 400 ["Failure"] No order is returned.
ListProducts(): Product[]				
listProducts(): Product[]	3	Empty	Products exist in IPOS-SA.	Status code: 200 ["Success"] A list of products are returned.
listProducts(): Product[]	3	Empty	Products do not exist in IPOS-SA.	Status code: 200 ["Success"] Nothing is returned, this is a false success on IPOS-SA's subsystem that must be checked by IPOS-PU manually.

7 Appendices

7.1 Initial statement of requirements by the client

This subsystem offers an online portal which a merchant may use to sell goods to members of the public online.

This subsystem should offer the following functionality:

IPOS-PU-MEMBERS: The package will allow members of the public to submit a membership application with the online portal. The applicant may select to register as either a "commercial" or a "non-commercial" customer. An application for a commercial membership should include a company registration number (with Companies House in the UK) and additional details on the Company Director(s), type of business, address, email address, etc. Applications for commercial membership will be passed to IPOS-SA. A decision will be taken by InfoPharma Ltd staff after conducting the necessary diligence checks. If the checks are successfully passed, the application for commercial membership will be approved. InfoPharma Ltd will notify the approved applicant by email, in which details on how to access IPOS – SA will be included. The customer may opt to receive the confirmation by regular mail (e.g. via a registered delivery).

Applications for non-commercial membership do not require any personal details to be provided. A decision on the application is taken by the portal after checking the validity of the email address provided by the applicant in the application. In case of successful registration, non-commercial members will be notified by an email in which their login details will be

included: the email provided by the applicant will be used as username, while a password, necessary for the login, will be generated by IPOS-PU (a random string of 10 symbols which include letters, numbers and special symbols). The newly registered non-commercial member will be forced to change their password when they login to IPOS_PU for the first time. Non-commercial members will be offered an incentive of 10% discount on every 10th order they make via the IPOS-PU portal.

IPOS-PU-SALES: The package will allow customers to view the catalogue of goods offered by the respective merchant. Filtering of the catalogue should be provided (e.g., to constrain the list to goods to those that match a keyword, e.g. “paracetamol”) to narrow down the list of products of interest to the customer. The customer will be able to select from the catalogue the products they require and place them in a shopping cart, confirm the order and checkout paying for them using a credit/debit card. A successful purchase will lead to an email being sent to the customer with details about how to track the order (from being received, to being dispatched and then delivered). The platform should maintain the status of the order until it is eventually delivered. Successful orders via IPOS-PU should be “propagated” to the respective merchant (i.e. IPOS-CA): the merchant’s inventory will be affected (deducting the amount of ordered goods).

IPOS-PU-COMMS: This package will offer communication with 3rd party services such as i) an email provider, e.g. Google SMTP, and ii) a Payment Processor, such as PayPal. The integration with the email service will be used by IPOS-PU and will be also made available to the other IPOS subsystems – IPOS-SA and IPOS-CA, to use as necessary. For instance, IPOS-SA will be emailing successful applicants the details of how they can use the respective IPOS-SA services.

IPOS-PU-PRM: This package will allow the respective merchant to run “promotion/advertising campaigns” via the IPOS-PU portal. The campaigns will be setup and managed by the Admin of the IPOS-PU subsystem. Each promotion campaign will be set to run from “Start Date/Time” to “End Day/Time” and will include a list of pharmaceutical goods from the Merchant’s stock with a fixed discount attached to each item. The same discount can apply to all items included in the promotion campaign, or different discounts can be applied to different items on the list. More than one promotion campaign can be “run” (i.e. be active) simultaneously, as necessary. A promotion campaign runs until the end of the specified period unless it is cancelled by the IPOS-PU Admin. Every time a member of the public visits IPOS-PU, they will be offered a link “Promotions”, provided the visit falls within one of the active promotion campaign periods. Clicking on this link will list the active promotion campaigns. The customer can then select one of the active promotion campaigns and look at the products included in it and the discounts offered

for different products. The customer can also proceed and compile a new order using the items on the list of products included in the campaign. The package will keep track of the “clicks” of links to each advertising campaign and the items included in the campaign. More specifically, if an advertising campaign CAMP_X contains a list of two items (Item_X_1 and Item_X_2), then IPOS-PU should maintain the following 5 counters: CAMP_X_counter of how many times the link to CAMP_X is clicked by customers visiting IPOS-PU, CAMP_X_1 and CAMP_X_2 counters which will be increased every time a customer includes Item_X_1 and Item_X_2, respectively, in an order, and finally, CAMP_X_1_purchased and CAMP_X_2_purchased, which count the number of items Item_X_1 and Item_X_2 included in completed orders during CAMP_X campaign. For instance, if an order includes 3 items of Item_X_1, then the counter CAMP_X_1 is increased by 3. Likewise, if an order includes 5 items of Item_X_2, then the counter CAMP_X_1 is increased by 5. Should the order be paid for, then CAMP_X_1_purchased and CAMP_X_2_purchased will be increased, too, by 3 and 5 respectively, but will not be increased if the order remains unpaid. Thus, CAMP_X_1 and CAMP_X_2 may be significantly greater than CAMP_X_1_purchased and CAMP_X_2_purchased, respectively. The ratios CAMP_X_1_purchased / CAMP_X_1 and CAMP_X_2_purchased / CAMP_X_1, respectively can be seen as a rough estimate of the “conversion rate” for each of the items included in a promotion campaign. Note that these counters may be significantly greater than the quantity of items which are actually purchased via IPOS-PU since only a fraction of the orders including items at promotional prices will be completed. Tracking the user behaviour (conversion rate), and how successful a particular promotion campaign has been, is achieved via a number of reports listed in the next package.

IPOS-PU-RPT: This package shall allow Admin to generate the following reports:

- Sales within a specified period (quantity of goods sold, unit price per unit and price for the quantity of sales per item).
- Advertising campaigns for a given period of time.
- Count of hits of advertising campaigns broken down by goods included in the advertising campaign.

7.2 Hardware (minimal and optimal configuration)

7.2.1 Minimal hardware configuration

The minimal hardware configuration recommended for IPOS-PU development consists of:

- Desktop PC or laptop capable of running a Java based desktop application
- 4 GB RAM
- At least 20 GB of free disk space for:

    - Application source code
    - Compiled program files
    - Local database storage (e.g. MySQL or similar DBMS)
- Operating System:
    - Windows 10 or later
- Java Runtime Environment (JRE) compatible with the implemented version
- Basic internet connection for:
    - Communication with external subsystems (IPOS-SA and IPOS-CA)
    - Email/payment service simulation

7.2.2 Optimal hardware configuration

The optimal hardware configuration envisaged for smoother development and testing consists of:

- Desktop PC or high performance laptop
- 8 GB RAM or higher
- At least 100 GB of free disk space for:
    - Source code and documentation
    - Larger test datasets
    - Local database server (e.g. MySQL, PostgreSQL, or similar)
- SSD storage recommended for improved performance
- Operating System:
    - Windows 11
- Latest stable Java Development Kit (JDK)
- Stable broadband internet connection for reliable subsystem communication

7.3 RESTful API Best Practises

The article linked here references some best practises for a REST API framework that the teams have aligned on – it delves into the formatting of HTTP requests, as well as a list of status codes that may be used to respond to requests. This also includes versioning of the API, and creating logs to monitor requests, to ensure accurate diagnosis of issues between subsystems.

REST APIs use the **Status-Line** part of an HTTP response message to inform clients of their request's overarching result. RFC 2616 defines the Status-Line syntax as shown below:

Status-Line = HTTP-Version SP Status-Code SP Reason-Phrase CRLF

RFC 2616 also defines the agreed upon conventions for how status codes should be used, but this documentation is intended for more technical audiences only.

Exact implementation of the REST API within the context of the IPOS software is to be planned further during the later design/implementation phase of development.